

Contributing to Open Source

Comprehensive Course Notes

Pluralsight | May 2026

1. Why Contribute to Open Source?

Before diving into the mechanics of contributing, it is worth asking a fundamental question: why bother at all? Open source software is not just free-of-charge — it is software built in the open, with its source code available for anyone to read, use, modify, and distribute. When you contribute to it, you become part of a global movement that powers the internet, hospitals, scientific research, and everyday applications.

Crucially, open source is chosen by organizations not because it is cheaper — though that helps — but because it is genuinely better software. It is battle-tested, peer-reviewed, and constantly improved by a community of motivated contributors. That is the environment you are joining.

1.1 Personal Growth

One of the most underrated aspects of open source is how much it stretches you as a person. When you fix a bug in a project used by thousands of people, you make a real, tangible impact — that feeling is difficult to replicate in a classroom or a side project nobody sees.

Beyond impact, contributing forces you to read unfamiliar code, understand foreign conventions, write clear technical communication, and adapt to an established workflow. These are precisely the skills that separate good engineers from great ones. The course highlights three real-world examples of open source projects serving communities:

- Hospital run systems — open source software managing critical patient workflows in healthcare settings
- Census Reporter — helps journalists and researchers interpret US census data clearly
- Refugee Restrooms — helps transgender people locate safe restrooms, built entirely by volunteers

1.2 Professional Growth

Your GitHub profile is a living, verifiable portfolio. Unlike a resume bullet point that says 'worked on large-scale systems,' open source contributions are public, linkable, and auditable. Employers increasingly check GitHub profiles before and during interviews.

Contributing also teaches you how professional engineering teams actually work: asynchronous communication, code reviews, branching strategies, CI/CD pipelines, and release cycles. These skills transfer directly to any job. Many companies actively hire contributors they have already worked with on open source — it is essentially a months-long public interview that happens in the open.

- Portable work experience — your contributions follow you regardless of which employer you work for
- Hiring pipeline — maintainers frequently offer roles to contributors whose work and judgment they trust
- Real teamwork skills — learn to collaborate effectively across time zones and cultures

- Interpersonal growth — writing constructive reviews, handling critical feedback gracefully, resolving disagreements professionally
 - Higher market value — demonstrable, verifiable skills command stronger offers and more interesting opportunities
-

2. Types of Contributions

A common misconception is that you can only contribute by writing code. In reality, thriving open source projects need a wide variety of contributions. The course organizes them into three broad categories: Planning, Social, and Development. Understanding this spectrum helps you find where you can add the most value right now, regardless of your current technical skill level.

2.1 Planning Contributions

Planning contributions help shape the direction of a project. This includes participating in roadmap discussions, triaging incoming issues — labeling them correctly, reproducing reported bugs, and closing duplicates — writing or refining feature proposals, and helping prioritize what the team works on next.

These contributions are extremely valuable because maintainers are often overwhelmed by volume. A contributor who reliably triages new issues, adds clear reproduction steps, or confirms whether a bug still exists on the latest version saves the maintainer enormous time — sometimes more time than a code fix would.

2.2 Social Contributions

Social contributions include anything that strengthens the community around the project. The most impactful forms are: answering questions in issues, GitHub Discussions, Stack Overflow, or Discord; improving documentation such as README files, wikis, tutorials, and API references; translating content into other languages; and creating blog posts, conference talks, or tutorial videos.

Documentation is chronically under-resourced in open source. If you find a concept confusing when first learning a project, many others do too. Writing a clear, accurate explanation of that concept — even just a paragraph added to the README — is a genuine and valued contribution that helps everyone who comes after you.

2.3 Development Contributions

Development contributions are what most people think of first: touching the source code. But this is broader than it sounds. It includes fixing bugs (especially those labeled 'good first issue'), implementing new features, writing or improving automated tests, improving build tooling or CI configuration, and reviewing other contributors' pull requests.

Even within code contributions, starting with small and tightly focused changes is the recommended strategy. A one-line fix that passes all tests and comes with a clear explanation is

infinitely more likely to be merged quickly than a sprawling pull request touching dozens of files across multiple concerns.

□ Key insight

You do not need to be a senior engineer to contribute meaningfully. Fixing a typo, answering a beginner question in the issue tracker, or translating the README are all real contributions that maintainers genuinely value.

3. Finding the Right Project

Choosing the right project is almost as important as the contribution itself. Contributing to a project that is a poor fit — wrong technology stack, hostile community, or an unmaintained codebase — can be demoralizing and unproductive. Taking 15 minutes to vet a project before investing significant effort is time well spent.

3.1 Ways to Explore Projects

- GitHub Explore (github.com/explore) — trending repositories filterable by language, topic, and time period
- Topics — search by technology tag such as 'react', 'machine-learning', or 'accessibility'
- Collections — curated lists around themes like 'open journalism', 'social impact', or 'education'
- Your own dependencies — the open source libraries your projects already depend on; you already know the API
- goodfirstissue.dev — aggregates beginner-friendly issues from thousands of GitHub repositories

3.2 Checking the Pulse of a Project

Before committing time, audit the project health using the following signals. Each red flag you find should make you reconsider your investment:

Signal	What to Look For
Recent commits	Is the main branch actively updated? No commits in 6+ months is a strong warning sign.
Issue response time	Do maintainers respond to issues within days or weeks? Months-old unanswered issues suggest abandonment.
CONTRIBUTING.md	Does a contributing guide exist? Without it you have no idea how the project expects work to be structured.
Good first issues	Labels like 'good first issue' or 'help wanted' signal a project actively welcoming newcomers.

Code of Conduct	Absence of a CoC means no stated norms for respectful interaction exist.
PR merge rate	Are pull requests being reviewed and merged in a reasonable time? A huge backlog of stale PRs is a warning sign.
Community tone	Read through issue and PR comments. Are interactions respectful and constructive, or dismissive and hostile?

❑ Red flags

No CONTRIBUTING.md, months of unanswered issues, dismissive comments, or a large backlog of unreviewed PRs all suggest your contribution will not be valued. Walk away — your time and energy are worth protecting.

4. Onboarding to a New Project

Even after finding a great project, there is a right way and a wrong way to get started. Jumping straight into writing code before reading the project's documentation is the single most common mistake new contributors make. Every project has its own norms, workflows, and expectations — reading the onboarding documents first helps you match those expectations from day one, and immediately marks you as a contributor worth taking seriously.

4.1 CONTRIBUTING.md — Your Primary Guide

This file is the single most important document in any open source repository. It typically covers: how to set up the development environment locally, what the preferred branching strategy is, how to run the test suite, what the code style guide requires, how to format commit messages, and what the pull request review process looks like.

If the project has a CONTRIBUTING.md, read it from top to bottom before writing any code. Maintainers can tell immediately when a contributor has not read it — submitting a PR with the wrong commit format, missing required test coverage, or not filling in the PR template are all signs of someone who skipped this step.

4.2 Code of Conduct

The Code of Conduct (CoC) defines the standards for respectful participation in the project community. Most projects adopt the Contributor Covenant as their CoC. Reading it tells you the expectations around communication, inclusivity, and how conflicts are handled.

Violating the CoC — even unintentionally — can result in your contributions being rejected or being permanently banned from the project. Read it carefully and take it seriously even if it seems like boilerplate at first glance.

4.3 CLA — Contributor License Agreement

Some larger projects, especially those backed by companies, require contributors to sign a Contributor License Agreement before their code can be merged. A CLA grants the project owner a legal license to use and distribute your contribution. Critically, it does not transfer your copyright — you still own the code you wrote — but it gives the project the rights it needs to include your work in their distribution.

Common CLA tools include CLA Assistant, a GitHub app that prompts you to sign by leaving a comment on your first pull request. If a project requires a CLA and you skip it, the CI bot will automatically block your PR from merging until the agreement is signed.

□ Onboarding order

Read CONTRIBUTING.md first. Then the Code of Conduct. Then check for a CLA requirement. Only after completing all three should you begin setting up the development environment and hunting for issues to work on.

5. Preparing to Make a Contribution

5.1 Start Small — The Power of Quick Wins

New contributors often feel pressure to make a big, impressive contribution on day one. Resist this urge. A small, clean fix teaches you the project's workflow, establishes a track record of reliability, and builds trust with maintainers. Once maintainers trust you, larger contributions get reviewed and merged much faster because you have already demonstrated good judgment.

Look for issues labeled 'good first issue', 'beginner', or 'help wanted'. These are intentionally kept for newcomers and are usually scoped to a single, well-defined change with clear success criteria — exactly the kind of issue where you can succeed and learn simultaneously.

5.2 Understanding Forks

In open source, you almost never push directly to the main repository. Instead, you create a fork — a personal copy of the repository under your own GitHub account. You have full write access to your fork and can push freely. When your work is ready, you open a Pull Request from your fork back to the original repository, which by convention is called 'upstream'.

This model is what makes open source scale: anyone can experiment freely on their own fork without ever having write access to the original, protecting the upstream from unauthorized changes while keeping contributions open to everyone.

5.3 Cloning Your Fork and Setting Up Remotes

After forking on GitHub, clone your fork to your local machine. Then add a reference to the original upstream repository so you can pull in its updates:

```
git clone https://github.com/YOUR-USERNAME/REPO-NAME.git
cd REPO-NAME
```

```
# Add the original repo as 'upstream'
git remote add upstream https://github.com/ORIGINAL-OWNER/REPO-NAME.git

# Verify both remotes are configured correctly
git remote -v
# origin      https://github.com/YOU/REPO.git  (fetch)
# origin      https://github.com/YOU/REPO.git  (push)
# upstream    https://github.com/ORIGINAL/REPO.git  (fetch)
# upstream    https://github.com/ORIGINAL/REPO.git  (push)
```

5.4 Keeping Your Fork Synchronized

The upstream repository receives new commits from many contributors every day. If your fork falls behind, your future PRs will have merge conflicts that are difficult and time-consuming to resolve. Syncing your fork before starting any new work is a habit worth building:

```
# Download the latest changes from upstream (no merge yet)
git fetch upstream

# Switch to your main branch
git checkout main

# Rebase your local main onto the latest upstream main
# (rebase is preferred over merge — it avoids extra merge commits)
git rebase upstream/main

# Push the updated main back to your fork on GitHub
git push origin main
```

5.5 Creating a Feature Branch

Never work directly on your fork's main branch. Always create a dedicated branch for each issue or piece of work. This keeps your main branch clean, makes it easy to keep in sync with upstream, and lets you work on multiple issues simultaneously without them interfering with each other.

```
# Create and immediately switch to a new branch
git checkout -b fix/issue-123-button-hover

# Common naming conventions used by many projects:
# fix/issue-123-short-description      (bug fixes)
# feat/add-dark-mode                  (new features)
# docs/update-installation-guide      (documentation)
# chore/upgrade-dependencies          (maintenance work)
```

6. Finding Work & Engaging the Team

6.1 Using GitHub Labels to Find Issues

Most projects use GitHub issue labels to categorize and prioritize work. Learning to filter and read these labels is the fastest path to finding work that is both appropriate for your skill level and actually wanted by the maintainers:

Label	What It Means
good first issue	Specifically reserved for newcomers. Well-scoped, with clear success criteria and usually some guidance.
help wanted	The maintainer wants a contribution here but is not actively working on it themselves.
bug	A confirmed problem in the current code, often with reproduction steps already documented.
documentation	Improvements to written content — may not require any code changes at all.
enhancement	A new feature request or improvement to an existing capability.
triage	Needs maintainer review — not yet confirmed, prioritized, or assigned.

6.2 Signaling Intent Before Starting Work

One of the most important social norms in open source is to signal your intent to work on an issue before you start writing code. This one habit prevents two contributors from unknowingly spending days on the same problem — a situation that wastes everyone's time and creates an awkward situation when one PR must be closed without being merged.

The right approach: comment on the issue saying something like 'Hi, I would like to take a stab at this. My plan is to [brief description of approach]. Does that sound like the right direction?' Then wait for a maintainer to acknowledge your comment, especially on high-visibility issues. Some projects formally assign issues — look for a maintainer saying 'Assigning to @you' before you start.

6.3 Using the GitHub UI for Quick Edits

For very small changes — fixing a typo, correcting a broken link, improving a sentence in the docs — you can skip cloning entirely. GitHub's built-in editor handles the entire workflow in your browser:

- Navigate to the file on GitHub and click the pencil (Edit this file) icon in the top right
- Make your change directly in the browser editor
- Scroll to 'Commit changes', select 'Create a new branch for this commit and start a pull request'

- Click 'Propose changes' — GitHub forks the repo if needed and opens a draft PR automatically

6.4 Opening a Draft PR Early

A Draft Pull Request is a PR explicitly marked as not yet ready for review. It is one of the most underused tools in the open source contributor's toolkit. Opening a draft PR within hours of starting work serves several purposes: it signals that the issue is being actively worked on, it invites early architectural feedback when it is cheapest to incorporate, it allows CI checks to run against your partial work, and it makes your progress visible to the entire team.

To open a Draft PR: push your branch to GitHub, click 'New pull request', then click the dropdown arrow next to the green 'Create pull request' button and choose 'Create draft pull request'. When your work is complete, click 'Ready for review' to promote it to a regular PR and request reviewers.

Best practice

Open a draft PR within a few hours of starting work — even if you only have a single empty commit. This one action tells the team the issue is claimed, prevents duplicate effort, and opens the door for early guidance from maintainers.

7. Drafting a Great Pull Request

The pull request is your primary communication artifact as a contributor. A great PR is not just correct code — it is a complete, self-contained story that tells reviewers what changed, why it changed, how to verify it works, and what trade-offs were considered. Maintainers review many PRs; one that is clear and self-explanatory gets reviewed faster and merged with far less back-and-forth.

7.1 Anatomy of a Great PR

Element	Guidance
Title	Specific and action-oriented. 'Fix login redirect after session expiry' is clear. 'Fix bug' is not. Start with a verb: Fix, Add, Update, Remove, Refactor.
Description	Explain the problem being solved, the approach taken, and any trade-offs. Answer: What changed? Why? How can it be tested? What are the risks?
Issue link	Use 'Closes #123' or 'Fixes #123' in the description body. GitHub automatically closes the linked issue when the PR is merged.
Screenshots	For any UI change, always include before and after screenshots. Visual evidence conveys information faster than reading code.

Test plan	Describe exactly how you tested the change: which commands to run, which edge cases you verified, and what result confirms it works.
Scope	Keep the PR focused on a single logical concern. If you find unrelated issues while working, open separate PRs for them — mixed concerns delay reviews significantly.

7.2 Writing Good Commit Messages

Commit messages inside your PR are part of the permanent project history. Many projects follow the Conventional Commits standard. A good message has a short summary line (50 characters or fewer) and an optional body explaining the why behind the change:

```
# Format: type(scope): short imperative summary  
fix(auth): redirect to /login after session expiry
```

```
Previously, expired sessions caused a blank screen because the  
router did not handle 401 responses. Added a global Axios  
interceptor that redirects to /login and preserves the  
originally requested URL as a query parameter for post-login  
redirect.
```

```
Fixes #247
```

7.3 Allowing Maintainer Edits

When opening a PR from your fork, always leave the checkbox 'Allow edits from maintainers' checked. This enables project maintainers to push small commits directly to your branch — fixing a variable name, resolving a trivial conflict, or adjusting a test — without needing to ask you first. This simple checkbox frequently cuts days off the time between submission and merge, and is considered standard good etiquette.

8. Collaborating Effectively on Pull Requests

8.1 Understanding Conflict in Open Source

The course makes an important and nuanced point: conflict in open source is not inherently bad. An antagonistic state — where two competing ideas are in tension — is a normal and healthy part of technical collaboration. Disagreements about architecture, naming, or approach lead to better outcomes when resolved thoughtfully. The goal is to distinguish between constructive technical disagreement (healthy and expected) and personal antagonism (unhealthy and unacceptable under the Code of Conduct).

When a reviewer pushes back on your approach, assume positive intent by default. They are not rejecting you as a person — they are questioning a specific technical decision. The best

response is genuine curiosity: 'That is a good point. Could you say more about the trade-offs you see with this approach, versus the alternative you have in mind?'

8.2 Requesting a Code Review

Once your PR is marked as ready, formally request a review. On the PR page, find the 'Reviewers' panel in the right-hand sidebar. Click the gear icon and type the reviewer's GitHub username. GitHub notifies them by email and adds the PR to their review queue.

If you are unsure who to request, look at who reviewed similar PRs that were recently merged into the repository. You can also check the CODEOWNERS file if the project has one — it maps file paths to the people responsible for reviewing changes to those files.

8.3 Responding to Review Feedback

Working through review feedback effectively is a skill that significantly influences how quickly your PRs get merged. The key principles:

- Read every comment before making any changes — understand the full scope of feedback before writing a single line
- Address every single comment — either make the requested change, or leave a reply explaining your reasoning for keeping it as-is
- For GitHub Suggestions (where a reviewer proposes specific code inline), commit them directly with 'Commit suggestion', or batch multiple into one commit for cleanliness
- Click 'Resolve conversation' after addressing each comment to keep the discussion organized and signal that each item has been handled
- After addressing all feedback, re-request review from the same reviewers using the Reviewers panel so they know to take another look
- If a PR goes quiet for more than a week, leave a polite comment asking if anything else is needed — do not let it go stale

8.4 Leaving Code Reviews for Others

As you become a more established contributor, you will start reviewing others' pull requests too. Great review comments are specific (they point to a line and name the concern), constructive (they suggest an alternative or ask a question rather than just rejecting), and calibrated by severity — prefix minor style nits with 'nit:' so authors know what is a blocker versus a preference.

Tone matters enormously in text-only communication. 'We usually avoid mutation here because it makes testing harder' lands very differently from 'You should not mutate state here.' The first educates; the second criticizes.

❏ Common mistake

Letting a PR go silent for weeks without responding to feedback. If you need time — for personal reasons, for research, or for anything else — leave a brief comment letting the reviewer know. A PR without author activity for too long gets marked stale and eventually closed.

9. Git Power Moves — Undoing & Rewriting History

Open source contributions often require more advanced Git skills than typical day-to-day development. Understanding how to safely undo mistakes, clean up commit history, and rebase onto the latest upstream code is essential for working effectively in a shared repository. These operations feel intimidating at first but become second nature with practice.

9.1 Undoing Specific Lines — Interactive Patching

Sometimes you want to revert only certain lines within a file without affecting the rest of the file. Git's interactive patch mode lets you step through each 'hunk' — a block of contiguous changed lines — and decide individually what to keep or discard:

```
git checkout -p <filename>

# Git shows each changed hunk and prompts:
# Apply this hunk? [y,n,q,a,d,s,e,?]
#   y = yes, discard this change (revert to original)
#   n = no, keep this change
#   s = split into even smaller hunks for finer control
#   q = quit, leave remaining hunks unchanged
#   ? = show full explanation of all options
```

9.2 Undoing an Entire Commit Safely — git revert

When you need to undo a specific commit that has already been pushed to a shared branch, use `git revert`. Unlike `git reset`, `revert` does not rewrite history — it creates a new commit that applies the exact inverse of the original, leaving the original commit intact. This is the safe choice for any commit that others may already have fetched.

```
# Create a new 'inverse' commit for a specific hash
git revert abc1234

# Git opens your editor for the revert commit message
# Save and close to complete the operation

# To revert multiple commits, combine them into one new commit:
git revert --no-commit abc1234
git revert --no-commit def5678
git commit -m 'revert: undo broken authentication refactor'
```

9.3 Squashing and Rewording — Interactive Rebase

Interactive rebase is Git's most powerful history-editing tool. It lets you rewrite the local history of a branch before sharing it — squashing many small work-in-progress commits into one clean commit, reordering commits so they tell a logical story, editing commit messages for clarity, or splitting a large commit into smaller focused ones. The most common use case is cleaning up a messy branch before opening or updating a PR:

```
# Rewrite the last 4 commits interactively
git rebase -i HEAD~4

# Git opens an editor showing your commits oldest-first:
# pick alb2c3  Add login form markup
# pick e4f5a6  Add form validation
# pick c7d8e9  fix typo
# pick 0alb2c  Add unit tests

# To squash commits 2 and 3 into commit 1, change to:
# pick alb2c3  Add login form markup
# squash e4f5a6  Add form validation
# squash c7d8e9  fix typo
# pick 0alb2c  Add unit tests

# Commands available in the rebase editor:
# squash = combine with commit above, merge messages
# fixup   = combine with commit above, discard message
# reword  = keep commit but edit its message
# edit    = pause rebase to amend the commit
# drop    = delete the commit entirely
# reorder = change order by moving lines up or down
```

9.4 Amending the Last Commit

If you need to fix the most recent commit — you forgot to stage a file, the message has a typo, or you need to add a small change — use `git commit --amend`. It replaces the last commit with a new one. Because this rewrites history, you will need to force push afterward if the commit was already pushed:

```
# Stage any files you want added to the last commit
git add forgotten-file.js

# Amend — opens editor to optionally update the message too
git commit --amend

# Amend without opening the editor (keep the existing message)
git commit --amend --no-edit
```

9.5 Rebasing onto Upstream

Before asking for a review on your PR, rebase your feature branch onto the latest upstream main. This pulls in any new changes from the upstream repo and replays your commits on top

of them — giving reviewers a clean diff that shows only your changes, without unrelated merge commits or outdated context. It is also necessary to resolve any conflicts with work that has landed since you started your branch:

```
# Fetch the latest state from the upstream repository
git fetch upstream

# Switch to your feature branch
git checkout my-feature

# Replay your commits on top of the latest upstream main
git rebase upstream/main

# If conflicts occur, Git pauses at each conflict:
# 1. Open the conflicted file and resolve the <<<< markers
# 2. Stage the resolved version
git add resolved-file.js
# 3. Continue replaying the remaining commits
git rebase --continue

# If you need to cancel and start over:
git rebase --abort
```

9.6 Force Pushing Safely

After any history-rewriting operation — rebase, amend, squash — your local branch and the remote branch have diverged. A regular git push will be rejected with 'rejected: non-fast-forward.' You need to force push. Always use `--force-with-lease` instead of `--force`:

```
# SAFE option: fails if the remote has new commits since your last fetch
git push --force-with-lease origin my-feature

# DANGEROUS option: overwrites the remote branch unconditionally
git push --force origin my-feature # avoid — can destroy collaborators' work
```

The `--force-with-lease` flag adds a critical safety check: it verifies that the remote branch has not received any new commits since you last fetched. If a maintainer pushed directly to your branch — perhaps accepting your 'Allow maintainer edits' checkbox — the `--force-with-lease` push will fail safely, alerting you to fetch and incorporate their changes before pushing again.

10. Advanced PR Scenarios

10.1 Integrating Changes from Another Fork

Sometimes you need to build on work that another contributor has submitted but that has not yet been merged into the upstream. You can pull their branch directly by adding their fork as a named remote:

```
# Add their fork as a remote with a descriptive name
git remote add contributor https://github.com/THEM/REPO.git
git fetch contributor

# Option A: Check out their entire branch locally
git checkout -b their-feature contributor/their-feature

# Option B: Cherry-pick just specific commits onto your branch
git cherry-pick <specific-commit-hash>
```

10.2 Splitting a Commit into Smaller Ones

A commit that mixes multiple unrelated concerns should be split before a PR is opened. The cleanest approach: reset the commit (unstage all its changes while keeping them in the working tree), then use interactive staging to recommit the changes in logical, focused groups:

```
# Un-commit the last commit - changes stay in your working tree
git reset HEAD~1

# Stage only the first logical group of changes interactively
git add -p
git commit -m 'feat: add dark mode toggle component'

# Stage the second group
git add -p
git commit -m 'fix: correct contrast ratio in page header'
```

10.3 Stacked (Sequential) Branches

For large features that are easiest to review in stages, you can stack branches: branch B branches off branch A, and B's PR targets branch A rather than main. When PR A merges, PR B's diff automatically reduces to show only its own changes. This is called a stacked PR workflow and is commonly used in large, active projects:

```
# Branch A - first stage of the feature
git checkout -b feat/part-A main
# ... do work, open PR targeting main ...

# Branch B - second stage, built on top of A
git checkout -b feat/part-B feat/part-A
# ... do work, open PR targeting feat/part-A ...

# After part-A merges, rebase part-B directly onto main
git fetch upstream
git checkout feat/part-B
git rebase upstream/main
```

10.4 Passing Required Status Checks

Most active projects configure required CI checks — automated tests, linting, type checking, security scans — that must all pass before a PR can be merged. If checks fail, push a fix to your branch and they re-run automatically. Always read the full failure logs in the CI tab rather than just the PR summary — the detailed output usually pinpoints exactly which test or rule failed and why.

10.5 Merging, Branch Cleanup, and Closing Etiquette

When your PR is approved and all required checks pass, a maintainer will merge it using one of three strategies: a merge commit (preserves full history), squash merge (collapses all commits into one), or rebase merge (replays commits onto main linearly). After merge, click the 'Delete branch' button GitHub presents to keep the repository clean.

If you decide not to finish a PR — due to time constraints, a change in direction, or because someone else solved it more elegantly — close it gracefully. Leave a comment explaining the reason. This gives context to anyone who finds the closed PR later through search, and it shows respect for the reviewers who spent time on your work.

11. Staying Updated with GitHub Social Features

Once you have made your first contribution, staying engaged with the project and the broader open source community is how you deepen relationships, discover new opportunities, and continue growing. GitHub has a rich set of social features specifically designed to help contributors stay informed and connected.

11.1 GitHub Explore and Trending

GitHub Explore (github.com/explore) surfaces trending repositories filtered by programming language, topic, and time period — today, this week, or this month. Spending a few minutes here each week is an excellent way to discover what the community is excited about, find new projects to contribute to, and stay aware of emerging tools before they become mainstream.

11.2 Watching Repositories

Watching a repository subscribes you to its activity notifications. GitHub offers several watch levels: 'Participating and @mentions' (notified only when you are directly involved or tagged — recommended for most repos), 'All Activity' (every issue, PR, comment, and commit — useful for repos you are very active in), and 'Releases only' (just new version announcements — good for tracking tools you use but do not contribute to actively). Choose the level that keeps you informed without overwhelming your notifications.

11.3 Subscribing to Individual Issues

You can subscribe to any individual issue or PR to be notified of updates even if you have not commented on it. This is particularly useful for tracking high-priority bugs you plan to work on, or to be notified when a feature you have been waiting for gets merged. Click 'Subscribe' in the right sidebar of any issue or PR page to opt in.

11.4 Following People

Following a GitHub user causes their public activity — repositories they star, fork, or create, and PRs they open — to appear in your personal activity feed. Following active maintainers and respected contributors in your area of interest is one of the most effective ways to discover valuable new projects, observe how senior engineers approach problems and collaborate, and build the kind of ambient awareness of your field that separates engaged practitioners from passive ones.

Feature	How to Use It
GitHub Explore	Discover trending repositories; see what the open source world is building each week
Watch a repository	Receive notifications at a configurable level: releases, all activity, or mentions only
Subscribe to an issue	Track specific issue or PR updates without having commented on it yourself
Follow a user	See their public activity in your feed; find projects they find worth starring or contributing to
Star a repository	Bookmark for later reference and signal appreciation to the maintainers
Notification settings	Fine-tune email vs. in-app delivery and set per-repository notification levels

☐ Reduce noise

Use 'Participating and @mentions only' as your default watch level for most projects. You will still see everything relevant to you without being buried in unrelated discussion threads.

12. Git Quick-Reference Cheatsheet

A consolidated reference for the most commonly used commands across a complete open source contribution workflow, from initial setup through PR submission and cleanup.

Initial Setup

```
git clone https://github.com/YOU/REPO.git
```

```
git remote add upstream https://github.com/ORIGINAL/REPO.git
git remote -v      # confirm both 'origin' and 'upstream' are listed
```

Sync Fork and Start New Work

```
git fetch upstream
git checkout main
git rebase upstream/main      # sync local main with upstream
git push origin main         # update your fork on GitHub
git checkout -b feat/my-work  # create a focused feature branch
```

Stage, Commit, Push

```
git add -p                  # stage changes hunk by hunk (recommended)
git commit -m 'fix(scope): short imperative summary'
git push origin feat/my-work
```

Clean Up History Before PR

```
git rebase -i HEAD~N        # squash, reword, reorder, or drop last N commits
git commit --amend          # fix the very last commit only
git push --force-with-lease origin feat/my-work
```

Safe Undo Operations

```
git revert <hash>           # undo a commit safely (creates a new inverse
commit)
git checkout -p <file>      # undo specific lines interactively
git rebase --abort          # cancel an in-progress rebase cleanly
git reflog                  # find lost commits — the emergency recovery tool
```

Working with Other Contributors

```
git remote add other https://github.com/THEM/REPO.git
git fetch other
git cherry-pick <hash>      # apply one specific commit from their branch
```

Key Takeaways

These are the principles that consistently separate contributors whose PRs get merged from those who struggle. Return to this list whenever you feel stuck, frustrated, or unsure of your next step.

- Start small. A clean, focused, well-explained quick fix builds trust faster than an ambitious but rough one. The first contribution sets the entire tone of your relationship with the project.
- Signal intent. Always comment on an issue before starting work. This one habit eliminates wasted duplicate effort and immediately marks you as a thoughtful contributor.
- Read the documentation first. CONTRIBUTING.md, the Code of Conduct, and the CLA are not optional reading. They are the rulebook for participating in that project's community.
- Draft PRs early. Opening a draft PR right after starting work invites early feedback and proves the issue is actively being handled — saving everyone time.
- Keep PRs tightly focused. One logical change per PR. Mixed concerns in a single PR are the primary reason PRs sit unreviewed for weeks.
- Technical conflict is healthy and expected. Disagreement about approach leads to better outcomes when handled with curiosity and respect. Never take it personally.
- Rebase over merge. Rebasing your branch onto upstream main keeps the project history linear and your diff clean and easy to review.
- Always use --force-with-lease. Never use plain --force when force-pushing — the safer option protects collaborators' work from accidental overwrites.
- Be a good community citizen. Thank your reviewers, close abandoned PRs with an explanation, delete merged branches, and treat every interaction as a long-term relationship investment.
- Stay engaged beyond your own PRs. Watch repos, follow contributors, subscribe to key issues. Relationships in open source compound over time into mentorships, job offers, and lasting friendships.